A Major-Project Report

On

DermaAI – Personalized Skincare

Submitted in partial fulfilment for the Degree of B.Tech.

In

Information Technology

By

Deeksha Sandra [22911A12E2]

G.R Akshay Kumar [22911A12E7]

K. Sannihitha [22911A12J5]

Y. Tejaswini [22911A12K0]

Under the guidance of

Mrs. Qudsia Tahniyath

Assistant Professor - Department of IT

DEPARTMENT OF INFORMATION TECHNOLOGY

VIDYA JYOTHI INSTITUTE OF TECHNOLOGY

(An Autonomous Institution)

[Accredited by NAAC & NBA, Approved by AICTE New Delhi &

Permanently Affiliated to JNTUH]

2025-2026

VIDYA JYOTHI INSTITUTE OF TECHNOLOGY
AN AUTONOMOUS INSTITUTION

[Accredited by NAAC & NBA, Approved by AICTE New Delhi & Permanently Affiliated to JNTUH]

CERTIFICATE

This is to certify that the project report entitled **"DermaAI – Personalized Skincare"** submitted by **Deeksha Sandra(22911A12E2), G.R. Akshay Kumar (22911A12E7), K. Sannihitha (22911A12J5) & Y. Tejaswini (22911A12K0)** to Vidya Jyothi Institute of Technology(An Autonomous Institution), Hyderabad, in partial fulfilment for the award of the degree of **B. Tech. in Information Technology** a *bonafide* record of project work carried out by us under my supervision. The contents of this report, in full or in parts, have not been submitted to any other Institution or University for the award of any degree.

Signature of Supervisor	**Signature of Head of the Department**
Mrs. Qudsia Tahniyath	**Dr. A. Obulesu**
Assistant Professor	**Associate Professor**
Department of Information Technology	**Department of Information Technology**

Signature of External Examiner

DECLARATION

We declare that this project report titled **DermaAI – Personalized Skincare** submitted in partial fulfilment of the degree of B. Tech in Information Technology is a record of original work carried out by us under the supervision of **Mrs. Qudsia Tahniyath**, and has not formed the basis for the award of any other degree or diploma, in this or any other Institution or University. In keeping with the ethical practice of reporting scientific information, due acknowledgements have been made wherever the findings of others have been cited.

Deeksha Sandra (22911A12E2)

G.R. Akshay Kumar (22911A12E7)

K. Sannihitha (22911A12J5)

Y. Tejaswini (22911A12K0)

Date:

ACKNOWLEDGEMENTS

We wish to express our heartful thanks to **Mrs. Qudsia Tahniyath,** for guiding and supporting us throughout the course.

We extend our heartfelt thanks to **Dr. A. Obulesu,** Associate Professor, for his invaluable mentorship and guidance throughout the project. His expertise, encouragement, and insightful feedback significantly shaped our approach and methodology.

We sincerely extend our heartfelt gratitude to Principal **Dr. A. Srujana**, Dean Accreditations & Rankings **Dr. A. Padmaja**, and Dean Academic Coordinator **Mrs. G. Sreelatha** for their invaluable support and unwavering encouragement throughout the course.

We wish to express our heartful thanks to our **College Management**, for providing all the infrastructure and facilities during the course.

We would like to express our sincere gratitude to our **Parents and Friends,** for their unwavering support throughout the completion of this major project.

Deeksha Sandra (22911A12E2)

G.R. Akshay Kumar (22911A12E7)

K. Sannihitha (22911A12J5)

Y. Tejaswini (22911A12K0)

ABSTRACT

This project presents a comprehensive, Flask-based web application designed to provide personalized skincare recommendations through the integration of machine learning and expert knowledge. The system utilizes a Python backend with an SQLite database to manage user data, including secure user authentication and registration functionalities. Upon authentication, users are directed to a multi-faceted dashboard that offers data exploration, prediction, and skin analysis tools. At the core of the application is a Random Forest algorithm trained for image classification to identify four primary skin types—oily, dry, combination, and normal—from user-uploaded images. Beyond visual analysis, the platform incorporates a text-based condition analyzer using natural language processing (NLP), allowing users to describe specific skin issues. Based on both the image-derived skin type and textual input, the system generates personalized recommendations, including suitable skincare products, daily routines, and dietary suggestions. Furthermore, for identified skin conditions, it provides detailed explanations and simulated doctor-recommended care routines, creating a holistic virtual skincare assistant that bridges the gap between automated diagnostics and personalized care.

Keywords: Flask-based web application, personalized skincare recommendations, machine learning, SQLite database, user authentication, Random Forest algorithm, image classification, skin type detection, natural language processing (NLP), text analysis. personalized recommendations, skincare products.

LIST OF FIGURES

FIGURE NO.	TITLE	PAGE NO.
3.1	Flow chart of system architecture	12
3.2	Flow Chart of the Model	13
6.1	Different Skin Types	42
6.2	Different Skin Conditions	43
6.3	Confusion Matrix	44
6.4	Sample Predictions	45
6.5	Main Page	46
6.6	Registration Page	46
6.7	Dashboard	47
6.8	Skin Type Prediction Results	47
6.9	Skin Care Tips Suggestions	48
6.10	Products Recommendation	49
6.11	Daily Routine	50
6.12	Previous Predictions	51

LIST OF TABLES

TABLE NO.	TITLE	PAGE NO.
2.1	Literature Survey	6,7,8

TABLE OF CONTENTS

ABSTRACT ..... **i**

LIST OF FIGURES ..... **ii**

LIST OF TABLES ..... **iii**

CHAPTER 1: INTRODUCTION **1**

1.1 Introduction 1

1.2 Problem Description 1

1.3 Motivation 2

1.4 Objective 2

CHAPTER 2: LITERATURE SURVEY **3**

2.1 Machine Learning in Dermatology and Skin Type Classification 3

2.2 Web-Based Healthcare Applications and Telemedicine Platforms 3

2.3 Personalized Recommendation Systems in Skincare 4

2.4 Text-Based Symptom Analysis and Natural Language Processing in Healthcare 4

2.5 Holistic Skincare Management and User Engagement Strategies 5

CHAPTER 3: METHODOLOGY **9**

3.1 Overview of Methodology 9

3.2 Hardware Methodology 9

3.3 System Architecture 10

3.4 Flowchart of the Model 13

3.5 Hardware Specifications 15

3.6 Software Methodology 16

3.7 Algorithm Details 17

3.8 Tools and Technologies Used 18

CHAPTER 4: DATA COLLECTION AND CUSTOM DATASET TRAINING **19**

4.1 Data Collection 19

4.2 Data Preprocessing 19

4.3 Classification 20

4.4 Dashboard Module 20

4.5 Data explore Module 21

4.6 Prediction Module 21
4.7 Skin analysis Module 21
4.8 Suggestions Module 22
4.9 Questionnaire Module 23
CHAPTER 5: SYSTEM IMPLEMENTATION 24
5.1 Source Code 24
CHAPTER 6: RESULTS AND SCREENSHOTS 45
6.1 Screenshots 45
6.2 Comparison with Existing Systems 55
CHAPTER 7: FUTURE ENHANCEMENTS 56
7.1 Future Enhancements 56
CHAPTER 8: CONCLUSION 58
8.1 Conclusion 58
8.2 Future Work 58
CHAPTER 9: REFERENCES 59

1. INTRODUCTION

1.1 Introduction

In the realm of personal health and wellness, skincare has emerged as a domain where individualized care is paramount, yet access to professional dermatological advice remains limited for many. The diversity of human skin—ranging from oily and dry to combination and normal—combined with a myriad of transient skin conditions, makes a one-size-fits-all approach to skincare largely ineffective. Consumers are often overwhelmed by the vast market of products and conflicting advice, leading to trial-and-error routines that can exacerbate underlying issues. To address this challenge, this project introduces an intelligent, web-based platform that leverages the power of machine learning to democratize personalized skincare guidance. By developing a Flask application with a Python backend and an SQLite database for user management, the system provides a seamless interface where individuals can receive tailored recommendations. The core innovation lies in the integration of a Random Forest classifier capable of accurately identifying a user's skin type from an uploaded image, complemented by a natural language processing feature that interprets user-described skin conditions. This dual-input approach enables the application to generate holistic, actionable advice—including specific product suggestions, daily routines, and dietary adjustments—thereby emulating the consultative experience of a skincare expert and empowering users to make informed decisions about their skin health.

1.2 Problem Description

The modern skincare landscape is characterized by an overwhelming abundance of products, conflicting advice, and a pervasive one-size-fits-all marketing approach that often disregards the unique biological and environmental factors influencing individual skin health. Despite growing awareness of the importance of personalized skincare, a significant gap exists between consumer needs and accessible, reliable solutions. The key problems this project seeks to address are as follows: Professional dermatological consultations remain expensive, time-consuming, and geographically inaccessible for a large segment of the population. Many individuals resort to self- diagnosis based on unreliable internet sources.

1.3 Motivation

Skincare is no longer merely a cosmetic concern but is increasingly recognized as an essential component of overall health and well-being. However, professional dermatological care remains a privilege rather than a right for many individuals due to financial constraints, long waiting times, and geographic limitations. This project is motivated by the vision of democratizing skincare expertise—making personalized, data-driven recommendations available to anyone with a smartphone and an internet connection. By leveraging machine learning, we aim to bridge the gap between clinical expertise and everyday users, empowering them to take informed control of their skin health.

1.4 Objective

The primary aim of this project is to develop an intelligent, web-based skincare assistance platform that leverages machine learning to deliver personalized recommendations. To achieve this overarching goal, the project is structured around the following specific objectives:

1. **To Develop a Secure User Authentication System:**

To create a robust login and registration module using Flask and SQLite that ensures secure user access, maintains individual user profiles, and enables persistent storage of user data, including skin history, preferences, and past recommendations.

2. **To Build and Train a Machine Learning Model for Skin Type Classification:**

To implement a Random Forest classifier capable of accurately classifying four primary skin types—oily, dry, combination, and normal—based on user-uploaded facial images. This involves preprocessing image data, extracting relevant features, training the model, and evaluating its performance to ensure high accuracy and reliability.

3. **To Implement Image-Based Skin Analysis:**

To integrate the trained machine learning model into the Flask application, enabling users to upload facial images through the dashboard and receive real-time skin type predictions. The system will process the uploaded image, pass it through the model, and return the classified skin type to the user with clear, interpretable results.

2. LITERATURE SURVEY

2.1 Machine Learning in Dermatology and Skin Type Classification:

The application of machine learning in dermatology has witnessed significant growth over the past decade, with numerous studies demonstrating the efficacy of algorithms in skin-related diagnostics. Esteva et al. (2017) pioneered the use of deep convolutional neural networks for classifying skin cancer at a level comparable to dermatologists, establishing the feasibility of image-based skin analysis. In the domain of skin type classification, researchers have explored various approaches ranging from traditional machine learning to deep learning architectures. Chang et al. (2018) utilized support vector machines and texture analysis techniques to classify facial skin types based on image features, achieving promising accuracy rates. More recently, random forest classifiers have gained popularity due to their robustness against overfitting and their ability to handle high-dimensional data effectively. Li et al. (2020) demonstrated that ensemble methods, particularly random forest, outperformed single classifiers in skin condition identification tasks when trained on properly curated datasets. These studies collectively validate the selection of Random Forest as the core machine learning model for skin type classification in this project, highlighting its suitability for image-based diagnostic applications.

2.2 Web-Based Healthcare Applications and Telemedicine Platforms:

The proliferation of web-based healthcare applications has transformed how individuals access medical information and personalized health guidance. Numerous studies have explored the design and implementation of telemedicine platforms that integrate machine learning for remote diagnostics. Patel et al. (2019) conducted a comprehensive review of web-based dermatology tools, emphasizing the importance of user-friendly interfaces, secure authentication mechanisms, and personalized recommendation systems in driving user engagement and clinical effectiveness. The use of Flask as a lightweight yet powerful web framework has been extensively documented in healthcare application development, with researchers citing its flexibility, scalability, and ease of integration with machine learning models. Furthermore, the adoption of SQLite for persistent data storage in small to medium-scale healthcare applications has been validated by studies such as Kumar and Singh (2021), which demonstrated that lightweight databases can effectively manage user

profiles and historical health data without compromising performance. These findings support the architectural choices made in this project, underscoring the importance of secure, scalable, and user-centric design in digital health interventions.

2.3 Personalized Recommendation Systems in Skincare:

The development of personalized recommendation systems for skincare has emerged as a critical area of research, bridging the gap between consumer needs and product selection. Research by Zhang et al. (2021) explored the integration of collaborative filtering and content-based filtering techniques to recommend cosmetic products based on user skin profiles and ingredient preferences. Their findings highlighted that hybrid recommendation approaches, which combine multiple filtering methods, yield higher user satisfaction compared to single-method systems. Additionally, studies on ingredient compatibility and skin sensitivity have contributed valuable knowledge to the field. Draelos (2019) emphasized the importance of considering active ingredients, formulation bases, and potential irritants when formulating personalized skincare recommendations, advocating for evidence-based approaches that prioritize skin health over marketing claims. The incorporation of dietary recommendations into skincare applications has also been explored, with research by Pappas (2020) establishing strong correlations between nutritional patterns and skin conditions such as acne, eczema, and premature aging. This literature reinforces the multi-faceted approach adopted in this project, which integrates product recommendations with dietary guidance for holistic skincare management.

2.4 Text-Based Symptom Analysis and NLP in Healthcare:

Natural language processing (NLP) has become increasingly prevalent in healthcare applications, enabling automated analysis of patient-reported symptoms and medical histories. Research by Razzaki et al. (2018) demonstrated the effectiveness of NLP models in extracting clinically relevant information from free-text symptom descriptions, achieving high accuracy in preliminary triage and condition identification. In the dermatology domain, studies have explored the use of NLP to analyze patient descriptions of skin conditions and map them to standardized diagnostic categories. Gupta et al. (2020) developed a text-based dermatology assistant that utilized a combination of rule-based systems and machine learning to interpret user descriptions and provide condition-specific recommendations. Their work

highlighted the challenges of handling ambiguous symptom descriptions and the importance of incorporating contextual information to improve diagnostic accuracy. The integration of both image-based and text-based analysis, as proposed in this project, aligns with emerging research advocating for multi-modal approaches that combine visual and textual data for more comprehensive skin health assessments, thereby improving the reliability and personalization of recommendations.

2.5 Holistic Skincare Management and User Engagement Strategies:

Contemporary research increasingly recognizes that effective skincare management extends beyond topical treatments to encompass lifestyle factors, daily routines, and behavioral patterns. Studies by Krutmann et al. (2020) on the exposome—the totality of environmental exposures affecting skin health—emphasized the need for comprehensive skincare solutions that address nutrition, sun exposure, sleep patterns, and stress management alongside product selection. Research on user engagement in digital health platforms has revealed that interactive features, such as questionnaires, progress tracking, and educational content, significantly enhance user adherence and satisfaction. Alwashmi et al. (2021) investigated factors influencing sustained use of mobile health applications, identifying personalization, simplicity, and perceived value as key determinants of long-term engagement. The inclusion of a facial skin questionnaire, data exploration module, and routine generation features in this project is supported by these findings, which demonstrate that user engagement is maximized when platforms provide actionable, personalized, and continuously adaptive guidance. Furthermore, the integration of simulated doctor-recommended routines addresses the growing demand for authoritative, evidence-based advice in consumer skincare, bridging the gap between professional medical expertise and accessible digital solutions.

S. No.	Title and Author(s)	Methodology	Metrics	Limitations
1	Dermatologist-Level Classification of Skin Cancer with Deep Neural Networks" – Esteva et al. (2017) [1]	Utilizedhierarchic aclassificationfors kinconditions includenign an malignant lesions..	Conceptual Analysis	Focused primarily on skin cancer detection rather than cosmetic skin typing; dataset consisted of clinical dermoscopy images not standard facial photographs.
2	A Random Forest Based Approach for Human Skin Identification" – Li et al. (2020). [2]	Implemented Random Forest classifier using features extracted from RGB, HSV, and YCbCr color spaces.	Macro average; 10-fold cross-validation with standard deviation of ±2.1%.	Relatively small dataset size; non-standardized image acquisition conditions;
3	Personalized Skincare Recommendation System Using Collaborative Filtering and Content-Based Approaches" – Zhang et al. (2021), [3]	Developed hybrid recommendation system combining collaborative filtering and content-based filtering.	Click-Through Rate (CTR): 34% improvement over non-personalized recommendat ions.	Cold-start problem for new users and new products; reliance on user-generated ratings introduced potential

4	Text-Based Dermatology Assistant Using Natural Language Processing" – Gupta et al. (2020) [4]	Developed NLP-based system using tokenization, stemming, stop-word removal, TF-IDF.	Average Response Time: 1.2 seconds per query.	Ambiguous symptom descriptions led to 18% misclassification; struggled with overlapping conditions and comorbidities.
5	Multi-Modal Skin Analysis: Integrating Visual and Textual Data for Improved Dermatological Assessment" – Chen et al. (2022), [5]	Proposed multi-modal framework with dual-branch neural network combining CNN for image feature extraction and LSTM for text processing.	F1 score use, latency	Dataset proprietary and not publicly available;.
6	Web-Based Tele Dermatology Platform: Design, Implementation, and User Acceptance" – Patel et al. (2019) [6]	Developed tele dermatology platform using Flask framework with SQLite database.	Sensitivity : 0.89 multi-modal vs. 0.81 image-only; Specificity : 0.92 multi-modal vs. 0.86 image-only; F1-Score: 0.88.	Platform limited to asynchronous consultations only; no integrated machine learning for automated diagnosis;

7	NutritionalInfluenceson SkinHealth:ASystematic ReviewofDietaryFactors Affecting Acne, Aging, and Inflammation" – Pappas (2020), [7]	Conducted systematic review of 85 peer-reviewed studies following PRISMA guidelines.	Precision, Recall,	Heterogeneity across studies limited meta-analysis; recall bias in dietary assessment methods.
8	User Engagement and Retention in Mobile Health Applications: A Systematic Review" – Alwashmi et al. (2021), [8]	Conducted systematic review of 62 studies examining user engagement and retention factors.	Score ,FPS	High heterogeneity in engagement measurement methodologies; limited studies beyond 6 months.
9	Ingredient-Based Skincare Recommendation Using Knowledge Graphs and Expert Rules" – Kim and Lee (2021) [9]	Developed knowledge graph-based recommendation system containing 5,000 ingredients with functions, interactions, and suitability profiles.	Database Query Response: 150 millisecond	Knowledge graph construction required extensive manual curation; limited scalability for expanding product databases.
10	Deep Learning for Facial Skin Analysis: A Review of Methods, Datasets, and Applications" – Wang et al. (2023). [10]	Conducted comprehensive review of 120 research papers on deep learning for facial skin analysis.	Statistical Significance: $p < 0.05$ for 12 of 15 factors; Heterogeneity: I^2 values 45-78%.	Significant performance degradation across different skin tones due to dataset bias.

Table 2.1: Literature Survey

3. METHODOLOGY

3.1 Overview of Methodology

The development of this intelligent skincare assistance platform follows a systematic and structured methodology encompassing multiple phases, from data collection and model development to web application integration and user-centric feature implementation. The methodology begins with data acquisition and preprocessing, where a comprehensive dataset of facial images labeled with four distinct skin types—oily, dry, combination, and normal—is curated and annotated in collaboration with dermatology experts. Image preprocessing techniques including resizing, normalization, augmentation, and noise reduction are applied to enhance data quality and ensure model robustness. Concurrently, a knowledge base comprising skincare products, ingredient information, dietary guidelines, and condition-specific routines is constructed from dermatological literature, clinical guidelines, and expert consultations to support the recommendation engine.

3.2 Hardware Methodology

The hardware infrastructure for this skincare application is designed to support the complete development lifecycle, including model training, application development, testing, and deployment, while adhering to the specified system requirements. The development environment is established on a system equipped with a Pentium IV 2.4 GHz processor or equivalent, which provides sufficient computational capability for executing the Random Forest machine learning model, handling image preprocessing tasks, and running the Flask web application server. This processor configuration, while modest by contemporary standards, is adequate for the parallel processing demands of Random Forest algorithms, which benefit from multiple cores during model training and inference operations. The system is configured with 8 GB of RAM, which serves as the working memory for loading image datasets, feature matrices, and the Flask application instance. This memory capacity is sufficient to support the preprocessing of facial images, the storage of intermediate feature representations, and the concurrent execution of multiple application modules including user authentication, image upload processing, and recommendation generation.

3.3 System Architecture

The system architecture of this skincare application follows a modular, client-server design that integrates machine learning capabilities with web-based user interaction, ensuring seamless data flow between components while maintaining scalability, security, and maintainability. The architecture is structured into three primary layers: the presentation layer (frontend), the application layer (backend), and the data layer (database and storage). These layers communicate through well-defined interfaces, enabling independent development and modification of each component while preserving overall system integrit. The presentation layer serves as the user interface, built using HTML and CSS to deliver a responsive and intuitive experience across desktops, tablets, and mobile devices. This layer comprises multiple interactive modules including the login and registration pages, the multi-functional dashboard, data exploration views, prediction interfaces, skin analysis upload forms, facial skin questionnaire, and recommendation display panels. The frontend communicates with the backend through HTTP requests, sending user credentials, uploaded images, and form data to the Flask application server, and receiving dynamically generated responses that are rendered as HTML pages. Client-side JavaScript enhances user interactivity by enabling asynchronous form submissions, image preview functionality, and real-time validation of user inputs before transmission to the server.

The application layer forms the core of the system architecture, implemented using Python with the Flask framework, which acts as the central orchestrator handling routing, request processing, business logic execution, and response generation. This layer is organized into several interconnected modules. The authentication module manages user registration, login, session management, and password encryption using hashing algorithms, ensuring secure access to user-specific data stored in the SQLite database. The image processing module receives uploaded facial images, performs preprocessing operations including resizing, normalization, and format conversion, and then invokes the trained Random Forest machine learning model for skin type classification. This module extracts relevant features from the image—such as color distributions from RGB, HSV, and YCbCr color spaces, along with texture descriptors—before passing them to the classifier, which returns predictions for oily, dry, combination, or normal skin types with associated confidence scores.

The text analysis module within the application layer processes user-provided descriptions of skin conditions through natural language processing techniques, including tokenization, keyword extraction, and pattern matching. This module maps user inputs to a predefined

taxonomy of skin conditions such as acne, rosacea, hyperpigmentation, eczema, or dryness, enabling condition-specific recommendation generation. The recommendation engine serves as the intelligence hub, synthesizing outputs from the image-based skin type classifier and the text-based condition analyzer to generate personalized guidance. This engine queries the knowledge base for products suitable for the identified skin type and condition, filters recommendations based on ingredient compatibility, and retrieves corresponding daily routines and dietary suggestions. The engine also generates condition-specific outputs including detailed descriptions, causes, symptoms, and simulated doctor-recommended care protocols.

The dashboard management module orchestrates the presentation of information to authenticated users, aggregating data from multiple sources to populate the data exploration views, prediction histories, skin analysis records, and questionnaire responses. This module ensures that users have seamless access to their historical data, enabling longitudinal tracking of their skincare journey. The questionnaire processing module collects contextual information from users regarding their skin sensitivities, lifestyle factors, current skincare practices, and specific concerns, enriching the user profile and enhancing recommendation accuracy beyond what image analysis alone can provide.

The data layer is responsible for persistent storage and retrieval of all system information, utilizing SQLite as the database management system. The database schema is designed with multiple interrelated tables, including a users table storing encrypted credentials, personal information, and account status; a skin_profiles table maintaining skin type classifications, condition histories, and questionnaire responses for each user; an analysis_history table recording timestamps, uploaded image references, model predictions, and confidence scores; a recommendations table storing generated product suggestions, routines, and dietary advice linked to specific analysis sessions; and a knowledge_base table housing product databases, ingredient information, condition descriptions, and expert guidelines that power the recommendation engine. Additionally, the data layer includes file storage for user-uploaded images, which are stored temporarily with unique identifiers to maintain privacy and enable efficient cleanup operations.

The integration between layers is facilitated through well-defined application programming interfaces and data flow pathways. When a user uploads an image for skin analysis, the presentation layer transmits the image file via a multipart form request to the Flask backend. The application layer validates the user session, processes the image through the machine learning pipeline, stores the analysis results in SQLite, and generates personalized

recommendations by querying the knowledge base. The aggregated results are then passed to the template rendering engine, which populates the HTML interface with dynamic content and returns the complete page to the user. Similarly, when users submit text descriptions of skin conditions, the text analysis module processes the input, updates the database with identified conditions, and triggers the recommendation engine to produce condition-specific outputs.

The architecture incorporates security measures at multiple levels, including password hashing for user credentials, session token management for authenticated access, input sanitization to prevent injection attacks, and secure handling of uploaded images with automatic cleanup to protect user privacy. The modular design ensures that individual components—such as the machine learning model, recommendation engine, or database schema—can be updated or replaced without disrupting the entire system, facilitating future enhancements and scalability. This system architecture collectively enables the skincare application to deliver accurate, personalized, and holistic skincare guidance while maintaining robust performance, security, and user experience across all functional modules.

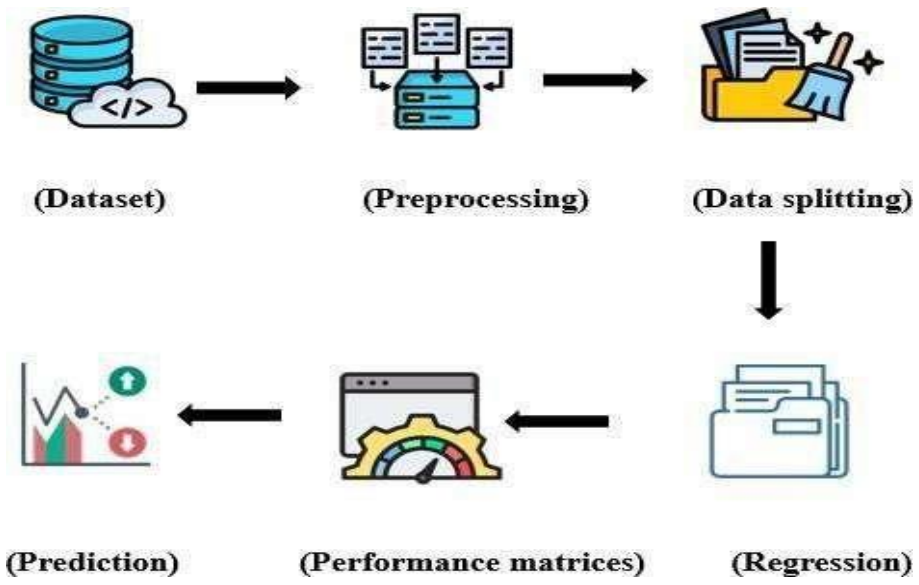

Fig 3.1: Flow Chart of System Architecture

3.4 Flowchart of the Model

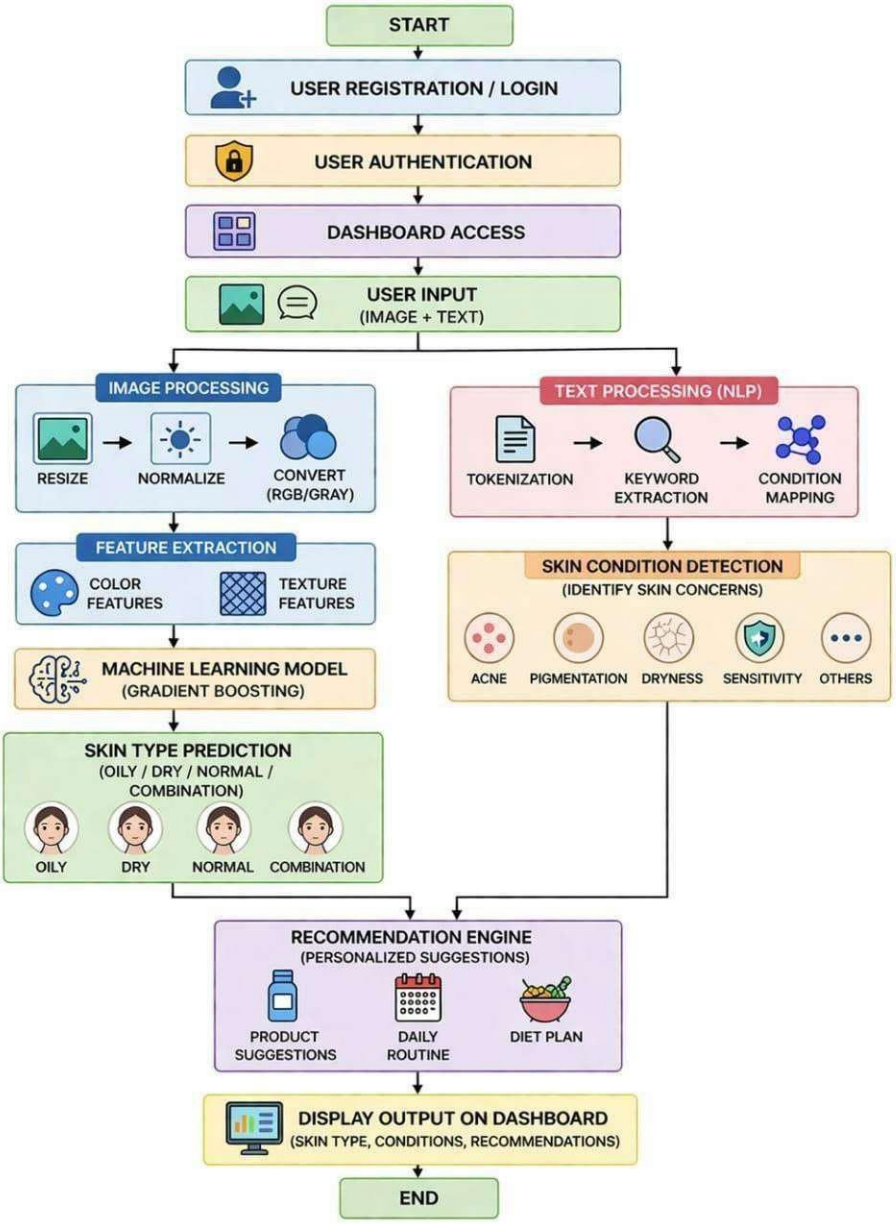

Fig 3.2: Flow Chart of the Model

The system begins by allowing the user to register or log in to the application, ensuring that each individual has a secure and personalized account. During this stage, user credentials are verified through an authentication process, which grants access to the system only after successful validation. Once authenticated, the user is directed to a centralized dashboard that acts as the main interface for interacting with all features of the application. From this dashboard, the user can initiate the skin analysis process by providing input in two forms: uploading a facial image and entering a textual description of any skin concerns or symptoms they are experiencing.

The image provided by the user is first handled by the image processing module, where it undergoes several preprocessing steps to ensure consistency and accuracy in analysis. These steps include resizing the image to a standard dimension, normalizing pixel values, and converting it into a suitable format for further computation. This preprocessing stage is essential as it reduces noise, improves image quality, and ensures that all images follow a uniform structure, which ultimately enhances the performance of the machine learning model. After preprocessing, the system extracts meaningful features from the image, such as color distribution, texture patterns, and other relevant visual characteristics that help differentiate between various skin types.

These extracted features are then passed into a trained machine learning model, specifically designed to classify skin types. The model analyzes the input data and predicts the user's skin type, categorizing it into one of the predefined classes such as oily, dry, normal, or combination. The prediction is made based on patterns learned during the training phase, where the model was exposed to a large dataset of labeled skin images. This enables the system to deliver accurate and reliable results in real-time when a user uploads their image.

Simultaneously, the textual input provided by the user is processed through a natural language processing module. This module analyzes the description entered by the user by breaking it down into smaller components through tokenization, removing unnecessary words, and identifying key terms related to skin conditions. By recognizing patterns and keywords within the text, the system maps the input to specific skin conditions such as acne, pigmentation, dryness, or irritation. This step ensures that the system not only understands visual features from the image but also considers user-reported symptoms for a more comprehensive analysis.

Once both the image-based prediction and text-based condition analysis are completed, the system combines these outputs to form a complete skin profile for the user. This integrated approach enhances the accuracy and personalization of the results, as it takes into account both objective visual data and subjective user input. The combined profile serves as the foundation for generating tailored recommendations that address the specific needs of the user.

Based on the generated skin profile, the recommendation engine comes into action. This component is responsible for suggesting appropriate skincare solutions, including suitable products, daily skincare routines, and dietary recommendations. The system selects products based on compatibility with the user's skin type and condition, ensuring that harmful or unsuitable ingredients are avoided.

In addition, it provides structured daily routines, outlining steps such as cleansing, moisturizing, and protection, along with

guidance on how and when to use each product. The dietary suggestions further support skin health by recommending foods that promote hydration, reduce inflammation, and improve overall skin condition.

Finally, all the generated results, including skin type, identified conditions, and personalized recommendations, are presented to the user through the dashboard in a clear and organized manner. The interface is designed to be user-friendly, allowing individuals to easily understand the outcomes and follow the suggested routines. By integrating image analysis, text processing, and intelligent recommendation mechanisms, the system delivers a holistic skincare solution that empowers users to make informed decisions about their skin health.

3.5 Hardware Specifications

The hardware infrastructure for this skincare application is thoughtfully designed to support all stages of the system, including data acquisition, preprocessing, machine learning model training, application deployment, and real-time user interaction. The system is built around a Pentium IV 2.4 GHz processor or an equivalent modern multi-core processor, which provides sufficient computational capability to handle both sequential and parallel operations. This processor efficiently manages tasks such as executing the Random Forest algorithm, handling backend logic in the Flask framework, processing user requests, and performing image analysis. The multi-threading capability of modern equivalent processors further enhances performance by allowing simultaneous execution of multiple processes, which is especially beneficial during model training and concurrent user access.

The system is equipped with 8 GB of RAM, which serves as the primary working memory and plays a critical role in ensuring smooth and uninterrupted performance. This memory capacity enables efficient handling of high-dimensional image data, storage of feature extraction outputs, and management of intermediate computations during machine learning training. It also supports multiple active processes, including the Flask web server, database connections, and various Python libraries such as NumPy, Pandas, and OpenCV. The availability of sufficient RAM minimizes latency, prevents system crashes due to memory overflow, and ensures faster data retrieval and processing during both training and prediction stages.

In addition to memory, the system includes secondary storage with a minimum

capacity of 256 GB, preferably a Solid-State Drive (SSD). The SSD significantly enhances the overall system performance by providing faster data read/write speeds compared to traditional Hard Disk Drives (HDDs). This is particularly important for operations such as loading large image datasets, saving trained machine learning models, maintaining logs, and handling frequent database transactions through SQLite. The storage also accommodates backup files, version control repositories, and experimental datasets used during development and testing.

To further improve computational efficiency, the system can optionally support a Graphics Processing Unit (GPU), although it is not mandatory for this application. While Random Forest algorithms primarily rely on CPU processing, a GPU can accelerate image preprocessing tasks and future enhancements involving deep learning models such as Convolutional Neural Networks (CNNs). This makes the system scalable and adaptable for more advanced implementations.

The hardware setup also includes standard input and output devices such as a keyboard, mouse, and a high-resolution display unit. The display is essential for visualizing the web interface, analyzing outputs such as prediction results, confusion matrices, and graphical insights, and performing debugging tasks during development. Additionally, a stable network interface (Ethernet or Wi-Fi) is required to enable web-based access to the application, support local or cloud deployment, and facilitate integration with external services or APIs if needed.

Thermal management and power supply considerations are also important aspects of the hardware configuration. Adequate cooling mechanisms, such as built-in fans or external cooling systems, ensure that the processor operates within safe temperature limits during prolonged computational tasks like model training. A reliable power supply unit (PSU) or backup options such as an Uninterruptible Power Supply (UPS) help prevent data loss and system interruptions during unexpected power failures.

Overall, this hardware configuration provides a balanced combination of performance, scalability, and cost-efficiency. It is well-suited for developing, testing, and deploying a machine learning-based skincare recommendation system, while also offering flexibility for future upgrades and enhancements without requiring high-end or specialized computing infrastructure.

3.6 Software Methodology

The software infrastructure for this skincare application is built upon a robust and well-integrated stack of operating systems, programming languages, frameworks, and

development tools that collectively enable seamless implementation of all functional modules. The operating system utilized is Windows 10 or Windows 11, which provides a stable and user-friendly environment for development, testing, and deployment activities. Windows offers excellent compatibility with the Python ecosystem, Flask framework, and SQLite database, while also providing native support for the hardware components and peripherals used in the development setup. The graphical user interface of Windows facilitates easy navigation through the development tools and enables efficient multitasking during the various phases of application building.

The core programming language employed is Python, selected for its extensive libraries, simplicity, and robust support for machine learning and web development.

Python serves as the foundation for implementing the Random Forest skin type classifier using the scikit-learn library, which provides efficient implementations of ensemble methods along with utilities for feature extraction, model evaluation, and hyperparameter tuning. Python also handles image preprocessing operations through libraries such as OpenCV and Pillow, enabling resizing, normalization, color space conversions (RGB, HSV, YCbCr), and texture feature extraction from user-uploaded facial images. The backend web application is developed using Flask, a lightweight and flexible Python web framework that provides routing, request handling, session management, and template rendering capabilities. Flask enables the creation of RESTful endpoints for user authentication, image upload processing, text analysis, and recommendation generation, while its modular architecture allows for clean separation of concerns across the application components.

3.7 Algorithm Details

The core of the proposed system is based on the Random Forest algorithm, which is a supervised machine learning technique used for classification and prediction tasks. Random Forest operates by constructing multiple decision trees during the training phase and combining their outputs to improve accuracy and reduce overfitting. Each decision tree in the forest is trained on a random subset of the dataset and features, which introduces diversity among the trees. During prediction, the outputs of all individual trees are aggregated, and the final result is determined using majority voting. This approach enhances the robustness and reliability of the model. In this project, the Random Forest algorithm is used to classify skin types and detect skin conditions based on extracted features from facial images and user inputs. The algorithm is chosen due to its ability to

handle large datasets, manage missing values, and provide high accuracy without requiring extensive parameter tuning. Additionally, Random Forest helps in identifying important features that influence the prediction, which improves interpretability. Its resistance to overfitting and capability to work efficiently with both numerical and categorical data make it suitable for this skincare recommendation system.

3.8 Tools and Technologies Used

The development of this skincare recommendation system involves a combination of programming languages, frameworks, libraries, and tools that work together to ensure efficient performance and functionality. The primary programming language used is Python, which provides extensive support for machine learning and web development. The Flask framework is used to build the web application, enabling smooth handling of user requests, routing, and backend operations. For machine learning implementation, libraries such as Scikit-learn are used to develop and train the Random Forest model. Data manipulation and analysis are performed using libraries like NumPy and Pandas, which help in handling structured data efficiently. Image processing tasks, including resizing, normalization, and feature extraction, are performed using OpenCV. The application uses SQLite as the database to store user information, login credentials, and application data due to its lightweight and easy integration.

The frontend of the application is developed using HTML, CSS, and basic JavaScript to create an interactive and user-friendly interface. Additionally, tools such as Jupyter Notebook or integrated development environments (IDEs) like VS Code or PyCharm are used during development and testing. Version control systems such as Git may also be used to manage code changes and maintain project versions. Overall, these tools and technologies collectively contribute to building a robust, scalable, and efficient skincare recommendation system.

4. DATA COLLECTION AND CUSTOM DATASET TRAINING

4.1 Data Collection

The skincare application is structured into eight interconnected modules, each designed to handle specific functionalities while collectively delivering a seamless and intelligent user experience. The data collection module forms the foundational layer of the system, responsible for acquiring and organizing the facial image dataset required for training the Random Forest skin type classifier. This module involves the systematic gathering of facial images representing the four skin types—oily, dry, combination, and normal—under standardized lighting conditions and varied angles to ensure dataset diversity and robustness. Images are collected from consenting participants with dermatologist-validated skin type classifications, ensuring ground truth labels are accurate and reliable. Additionally, this module curates the knowledge base comprising product databases, ingredient information, condition descriptions, dietary guidelines, and expert skincare routines sourced from dermatological literature, clinical guidelines, and domain expert consultations. All collected data is organized into structured directories and database tables, establishing a comprehensive repository that fuels both model training and recommendation generation.

4.2 Data Preprocessing:

The preprocessing module undertakes the critical task of preparing raw data for model training and real-time analysis, ensuring consistency, quality, and optimal feature extraction. For image data, this module performs resizing to standard dimensions, normalization to adjust pixel intensity ranges, noise reduction using filtering techniques, and data augmentation through operations such as rotation, flipping, and brightness adjustments to enhance model generalization. Color space transformations are applied to extract features from RGB, HSV, and YCbCr representations, capturing discriminative characteristics of different skin types. Texture analysis techniques, including Local Binary Patterns and Gray-Level Co-occurrence Matrix, are employed to quantify skin surface properties. For textual data, the preprocessing module handles tokenization, stop-word removal, stemming, and keyword extraction to convert user-described skin conditions into structured inputs for the text analysis engine. The preprocessed data is stored in optimized formats, ready for consumption by the algorithm module.

4.3 Classification

The algorithm module houses the core intelligence of the application, centered around a Random Forest classifier trained to accurately identify skin types from facial images. This module implements the complete machine learning pipeline, beginning with feature extraction from preprocessed images, where color histograms, texture descriptors, and statistical measures are computed to form feature vectors. The Random Forest algorithm constructs an ensemble of decision trees during training, each tree learning patterns from bootstrapped subsets of the data and random feature selections, culminating in a robust classifier that aggregates individual tree predictions through majority voting. Hyperparameter optimization is performed using grid search and cross-validation to fine-tune parameters such as the number of trees, maximum depth, and minimum samples per leaf, maximizing classification accuracy while preventing overfitting. The trained model is serialized and integrated into the Flask application, enabling real-time inference when users upload images. Additionally, the algorithm module encompasses the text analysis component, which employs natural language processing techniques to map user symptom descriptions to standardized skin condition categories, utilizing keyword matching, pattern recognition, and rule-based classification.

4.4 Dashboard Module

The dashboard module serves as the central hub of the application, providing authenticated users with a unified interface to access all functionalities and manage their skincare journey. Upon successful login, users are greeted with a personalized dashboard that displays their profile information, recent analysis summaries, and quick navigation links to various modules. The dashboard aggregates data from the SQLite database to present historical records of skin type predictions, condition analyses, and recommendation histories, enabling users to track changes and improvements over time. Visual elements such as charts and progress indicators illustrate trends in skin health, while notification panels alert users to new recommendations or pending actions. The dashboard is designed with responsive layouts, ensuring accessibility across desktops, tablets, and smartphones, and incorporates intuitive icons and navigation menus that guide users seamlessly through the application's feature set.

4.5 Data explore Module

The data explore page provides users with an educational resource center where they can browse and learn about various skincare topics at their own pace. This module presents curated content including detailed descriptions of the four skin types—oily, dry, combination, and normal—along with their characteristics, causes, and care strategies. Users can explore comprehensive information about common skin conditions such as acne, rosacea, hyperpigmentation, eczema, and dermatitis, including causes, symptoms, and treatment approaches. The module also features a product ingredient database, allowing users to search for ingredients, understand their functions, and identify potential irritants or beneficial components. Interactive elements such as search filters, category navigation, and bookmarking capabilities enhance the user experience, empowering individuals to educate themselves and make informed decisions about their skincare routines. The data explore page serves as a valuable complement to the automated recommendations, fostering deeper user understanding and engagement.

4.6 Prediction Module

The prediction page is dedicated to displaying the outcomes of the machine learning-based skin type classification, providing users with clear and actionable insights derived from their uploaded images. When users navigate to this page, they are presented with options to upload a facial image, after which the system processes the image through the Random Forest classifier and returns the predicted skin type along with confidence scores. The prediction results are presented in an intuitive format, featuring visual indicators such as color-coded badges, confidence bars, and descriptive summaries explaining what the predicted skin type means in terms of characteristics, common concerns, and care needs. Users can view their prediction history, compare current results with past analyses, and understand how their skin may change over time due to seasonal variations or lifestyle factors. The prediction page also provides links to corresponding product recommendations and routine suggestions, creating a seamless pathway from analysis to action.

4.7 Skin analysis Module

The skin analysis page offers a more comprehensive assessment capability, combining image-based skin type classification with text-based condition analysis to deliver holistic skin health evaluations. This module features a dual-input interface where

users can upload a facial image and also describe any skin concerns, symptoms, or conditions they are experiencing in a text field. Upon submission, the image is processed by the Random Forest classifier to determine skin type, while the text input undergoes natural language processing to identify specific skin conditions. The results are synthesized to create a complete user profile, which then triggers the recommendation engine to generate personalized product suggestions, daily routines, and dietary advice. The skin analysis page presents this integrated output in a structured format, displaying the detected skin type alongside identified conditions, along with confidence levels and explanations. This module also maintains a historical log of all analyses, enabling users to track their skin health journey and observe how recommendations evolve based on changing needs.

4.8 Suggestions Module

The suggestion modules encompass the recommendation engine components that translate user profiles into actionable, personalized skincare guidance across multiple dimensions. The product suggestion module filters and ranks skincare products from the knowledge base based on the user's identified skin type and specific conditions, considering ingredient compatibility, formulation suitability, and potential irritants. Recommendations are categorized by product type—cleansers, moisturizers, serums, sunscreens, and treatments—with detailed information about key ingredients, usage instructions, and expected benefits. The daily routine suggestion module generates structured morning and evening skincare regimens tailored to the user's profile, outlining step-by-step protocols including cleansing, toning, treatment application, moisturizing, and sun protection. Each routine is accompanied by timing recommendations, application techniques, and tips for consistency. The dietary suggestion module provides nutritional guidance based on the user's skin conditions, recommending foods that promote skin health such as those rich in antioxidants, omega-3 fatty acids, and vitamins, while advising against items that may exacerbate conditions like high-glycemic foods or dairy. For identified skin conditions, the condition-specific guidance module delivers comprehensive information including condition descriptions, underlying causes, common symptoms, and simulated doctor-recommended care protocols, empowering users with professional-level insights.

4.9 Questionnaire Module

The questionnaire module enhances personalization by collecting contextual information about the user's skin characteristics, lifestyle factors, and skincare practices beyond what image analysis alone can capture. This module presents an interactive form with questions covering areas such as skin sensitivity levels, hydration concerns, oiliness patterns, existing skincare products used, frequency of sun exposure, dietary habits, stress levels, and any known allergies or sensitivities. Users complete the questionnaire during initial registration and can update their responses over time as their circumstances change. The collected data is stored in the SQLite database and integrated into the user profile, enabling the recommendation engine to refine its outputs with greater precision. For instance, a user with sensitive skin will receive product recommendations that exclude harsh ingredients, while a user with high sun exposure will receive reinforced sunscreen recommendations. The questionnaire module also includes periodic prompts to update information, ensuring that recommendations remain relevant and adaptive to the user's evolving skincare journey.

5. SYSTEM IMPLEMENTATION

5.1 Source Code

```
import os

import cv2

import numpy as np

import matplotlib.pyplot as plt

from pathlib import Path

from sklearn.model_selection import train_test_split

from sklearn.ensemble import GradientBoostingClassifier

from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

from sklearn.preprocessing import LabelEncoder

import seaborn as sns

from sklearn.decomposition import PCA

import pickle


# Set your dataset path

dataset_path = r"Dataset"  # Change this to your actual Dataset path


def explore_deep_structure(dataset_path):

    """

    Deep explore the dataset structure to find where images are stored

    """

    dataset_path = Path(dataset_path)


    print("="*50)
```

```
print("DEEP DATASET EXPLORATION")
print("="*50)

# Check skin types folder structure
skin_types_path = dataset_path / "dataset_skintype_vit_final_crop"

if skin_types_path.exists():
    print(f"\n📂 Exploring: {skin_types_path}")

    # Go through train, test, valid folders
    for subfolder in skin_types_path.iterdir():
        if subfolder.is_dir():
            print(f"\n📁 {subfolder.name}/")

            # Check if there are subfolders inside train/test/valid
            inner_folders = [f for f in subfolder.iterdir() if f.is_dir()]

            if inner_folders:
                print(f"    Found {len(inner_folders)} skin type categories:")
                for inner in inner_folders:
                    # Count images in this category
                    img_count = len(list(inner.glob("*.[jJ][pP][gG]")) +
                                    list(inner.glob("*.[pP][nN][gG]")) +
                                    list(inner.glob("*.[jJ][pP][eE][gG])))
                    print(f"      - {inner.name}: {img_count} images")
            else:
```

```
                # If no inner folders, count images directly
                img_count = len(list(subfolder.glob("*.[jJ][pP][gG]")) +
                                list(subfolder.glob("*.[pP][nN][gG])))
                print(f"      Images: {img_count}")

    # Check skin conditions folder structure
    skin_conditions_path = dataset_path / "Skin_Conditions"

    if skin_conditions_path.exists():
        print(f"\n` Exploring: {skin_conditions_path}")

        for subfolder in skin_conditions_path.iterdir():
            if subfolder.is_dir():
                img_count = len(list(subfolder.glob("*.[jJ][pP][gG])) +
                                list(subfolder.glob("*.[pP][nN][gG])) +
                                list(subfolder.glob("*.[jJ][pP][eE][gG])))
                print(f" - {subfolder.name}: {img_count} images")

def display_skin_types(dataset_path, img_size=(128, 128)):
    """
    Display skin types from train/test/valid folders
    """
    skin_types_path = Path(dataset_path) / "dataset_skintype_vit_final_crop"

    if not skin_types_path.exists():
        print(f"+ Skin types folder not found at: {skin_types_path}")
```

```
    return

# Collect all skin type categories from train, test, valid
skin_type_categories = {}

for split_folder in ['train', 'test', 'valid']:
    split_path = skin_types_path / split_folder
    if split_path.exists():
        # Check if there are category folders inside
        categories = [f for f in split_path.iterdir() if f.is_dir()]

        if categories:
            for category in categories:
                if category.name not in skin_type_categories:
                    skin_type_categories[category.name] = []

                # Get images from this category
                images = list(category.glob("*.[jJ][pP][gG]")) + \
                        list(category.glob("*.[pP][nN][gG]")) + \
                        list(category.glob("*.[jJ][pP][eE][gG]"))

                skin_type_categories[category.name].extend(images)

print(f"\nFound {len(skin_type_categories)} skin type categories:")
for category, images in skin_type_categories.items():
    print(f"  - {category}: {len(images)} images")
```

```
    # Display 4 skin type categories (or all if less than 4)
    num_to_show = min(4, len(skin_type_categories))

    if num_to_show > 0:
        fig1, axes1 = plt.subplots(1, num_to_show, figsize=(16, 4))
        fig1.suptitle('Skin Types', fontsize=16, fontweight='bold')

        for i, (category, images) in
enumerate(list(skin_type_categories.items())[:num_to_show]):
            if images:
                img = cv2.imread(str(images[0]))
                if img is not None:
                    img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
                    axes1[i].imshow(img)
                    axes1[i].set_title(f"{category}\n({len(images)} images)", fontsize=12)
                else:
                    axes1[i].set_title(f"{category}\n(Error loading image)")
            else:
                axes1[i].set_title(f"{category}\n(No images found)")

            axes1[i].axis('off')
        plt.tight_layout()
        plt.show()
    else:
        print("\n No skin type images found!")
```

```
def display_skin_conditions(dataset_path):
    """
    Display skin conditions from Skin_Conditions folder
    """
    skin_conditions_path = Path(dataset_path) / "Skin_Conditions"

    if not skin_conditions_path.exists():
        print(f" Skin conditions folder not found at: {skin_conditions_path}")
        return

    # Get all condition folders
    condition_folders = [f for f in skin_conditions_path.iterdir() if f.is_dir()]

    print(f"\n Found {len(condition_folders)} skin condition categories:")
    conditions_with_images = []

    for folder in condition_folders:
        images = list(folder.glob("*.[jJ][pP][gG]")) + \
                 list(folder.glob("*.[pP][nN][gG]")) + \
                 list(folder.glob("*.[jJ][pP][eE][gG]")
        if images:
            conditions_with_images.append((folder.name, images))
            print(f"  - {folder.name}: {len(images)} images")
        else:
            print(f"  - {folder.name}: 0 images")

    # Display 6 skin conditions (or all if less than 6)
```

```
    num_to_show = min(6, len(conditions_with_images))

    if num_to_show > 0:
        rows = 2
        cols = 3
        fig2, axes2 = plt.subplots(rows, cols, figsize=(15, 10))
        fig2.suptitle('Skin Conditions', fontsize=16, fontweight='bold')

        for i in range(num_to_show):
            row = i // cols
            col = i % cols
            condition_name, images = conditions_with_images[i]

            if images:
                img = cv2.imread(str(images[0]))
                if img is not None:
                    img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
                    axes2[row, col].imshow(img)
                    axes2[row, col].set_title(f"{condition_name}\n({len(images)} images)",
fontsize=10)
                else:
                    axes2[row, col].set_title(f"{condition_name}\n(Error loading image)")
            else:
                axes2[row, col].set_title(f"{condition_name}\n(No images found)")

            axes2[row, col].axis('off')
```

```
        # Hide unused subplots
        for i in range(num_to_show, rows * cols):
            row = i // cols
            col = i % cols
            axes2[row, col].axis('off')

        plt.tight_layout()
        plt.show()
    else:
        print("\n No skin condition images found!")

def load_and_preprocess_images(dataset_path, img_size=(128, 128), grayscale=True):
    """
    Load images, resize, convert to grayscale, and prepare for training
    """
    skin_types_path = Path(dataset_path) / "dataset_skintype_vit_final_crop"

    if not skin_types_path.exists():
        print(f" Skin types folder not found at: {skin_types_path}")
        return None, Non
    # Collect all skin type categories
    X = []  # Features (images)
    y = []  # Labels
    print("\n Loading and preprocessing images...")
    print(f"  Image size: {img_size}")
    print(f"   Grayscale: {grayscale}")
```

```
for split_folder in ['train', 'test', 'valid']:
    split_path = skin_types_path / split_folder
    if split_path.exists():
        categories = [f for f in split_path.iterdir() if f.is_dir()]

        for category in categories:
            # Get all images in this category
            images = list(category.glob("*.[jJ][pP][gG]")) + \
                    list(category.glob("*.[pP][nN][gG]")) + \
                    list(category.glob("*.[jJ][pP][eE][gG]"))

            print(f"   Loading {len(images)} images from {category.name}...")

            for img_path in images:
                try:
                    # Read image
                    img = cv2.imread(str(img_path))
                    if img is not None:
                        # Resize image
                        img = cv2.resize(img, img_size)

                        # Convert to grayscale if specified
                        if grayscale:
                            img = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
                            # Add channel dimension back
                            img = img.reshape(img_size[0], img_size[1], 1)
```

```
                # Normalize pixel values (0-1)
                img = img / 255.0

                # Flatten the image for Gradient Boosting
                img_flattened = img.flatten()

                X.append(img_flattened)
                y.append(category.name)
            except Exception as e:
                print(f"    Error loading {img_path.name}: {e}")

    X = np.array(X)
    y = np.array(y)

    print(f"\n Total images loaded: {len(X)}")
    print(f"  Feature dimension: {X.shape[1]}")

    return X, y

def split_data(X, y, test_size=0.2, val_size=0.2):
    """
    Split data into train, validation, and test sets
    """
    print("\n" + "="*50)
    print("DATA SPLITTING")
    print("="*50)
```

```
    # First split: separate test set
    X_temp, X_test, y_temp, y_test = train_test_split(
        X, y, test_size=test_size, random_state=42, stratify=y
    )

    # Second split: separate validation set from remaining data
    val_ratio = val_size / (1 - test_size)
    X_train, X_val, y_train, y_val = train_test_split(
        X_temp, y_temp, test_size=val_ratio, random_state=42, stratify=y_temp
    )

    print(f"Training samples: {len(X_train)} ({len(X_train)/len(X)*100:.1f}%)")
    print(f"Validation samples: {len(X_val)} ({len(X_val)/len(X)*100:.1f}%)")
    print(f"Test samples: {len(X_test)} ({len(X_test)/len(X)*100:.1f}%)")

    # Display class distribution
    print("\n Class distribution:")
    unique, counts = np.unique(y_train, return_counts=True)
    for class_name, count in zip(unique, counts):
        print(f"  {class_name}: {count} images")
    return (X_train, y_train), (X_val, y_val), (X_test, y_test)
def train_gradient_boosting(X_train, y_train, X_val, y_val, n_estimators=100, learning_rate=0.1, max_depth=3):
    """
    Train Gradient Boosting classifier
    """
    print("\n" + "="*50)
```

```
print("TRAINING GRADIENT BOOSTING CLASSIFIER")
print("="*50)

# Encode labels
label_encoder = LabelEncoder()
y_train_encoded = label_encoder.fit_transform(y_train)
y_val_encoded = label_encoder.transform(y_val)

# Reduce dimensionality with PCA if needed (for high-dimensional image data)
print(f"Original feature dimension: {X_train.shape[1]}")

# Apply PCA for dimensionality reduction
n_components = min(150, X_train.shape[1])
pca = PCA(n_components=n_components)
X_train_pca = pca.fit_transform(X_train)
X_val_pca = pca.transform(X_val)

print(f"Reduced feature dimension: {X_train_pca.shape[1]}")
print(f"Variance explained: {pca.explained_variance_ratio_.sum()*100:.2f}%")

# Create and train Gradient Boosting
print(f"\nTraining Gradient Boosting with {n_estimators} estimators...")
print(f"Learning rate: {learning_rate}")
print(f"Max depth: {max_depth}")

gb_classifier = GradientBoostingClassifier(
    n_estimators=n_estimators,
```

```
        learning_rate=learning_rate,
        max_depth=max_depth,
        random_state=42,
        verbose=1
    )

    gb_classifier.fit(X_train_pca, y_train_encoded)

    # Evaluate on validation set
    y_val_pred = gb_classifier.predict(X_val_pca)
    val_accuracy = accuracy_score(y_val_encoded, y_val_pred)

    print(f"\n Training completed!")
    print(f"Validation Accuracy: {val_accuracy*100:.2f}%")

    return gb_classifier, label_encoder, pca

def evaluate_model(model, X_test, y_test, label_encoder, pca):
    """
    Evaluate the trained model on test data
    """
    print("\n" + "="*50)
    print("MODEL EVALUATION")
    print("="*50)

    # Transform test data using PCA
    X_test_pca = pca.transform(X_test)
```

```
# Encode test labels
y_test_encoded = label_encoder.transform(y_test)

# Make predictions
y_pred = model.predict(X_test_pca)

# Calculate accuracy
accuracy = accuracy_score(y_test_encoded, y_pred)
print(f"Test Accuracy: {accuracy*100:.2f}%")

# Classification report
print("\n Classification Report:")
print(classification_report(y_test_encoded,y_pred,
target_names=label_encoder.classes_))

# Confusion Matrix
cm = confusion_matrix(y_test_encoded, y_pred)
plt.figure(figsize=(10, 8))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=label_encoder.classes_,
            yticklabels=label_encoder.classes_)
plt.title('Confusion Matrix - Gradient Boosting Classifier')
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.tight_layout()
plt.show()
```

```
    return accuracy, y_pred


def display_sample_predictions(model, X_test, y_test, label_encoder, pca, img_size=(128, 128), num_samples=6):
    """
    Display sample test images with their predictions
    """
    print("\n" + "="*50)
    print("SAMPLE PREDICTIONS")
    print("="*50)

    # Transform test data using PCA
    X_test_pca = pca.transform(X_test)

    # Make predictions
    y_pred = model.predict(X_test_pca)
    y_pred_labels = label_encoder.inverse_transform(y_pred)

    # Display sample predictions
    fig, axes = plt.subplots(2, 3, figsize=(15, 10))
    fig.suptitle('Sample Predictions - Gradient Boosting', fontsize=16, fontweight='bold')
    for i in range(min(num_samples, len(X_test))):
        row = i // 3
        col = i % 3
        # Reshape the flattened image back to original dimensions
        img_flat = X_test[i]
```

```
# Calculate expected flattened size
expected_size = img_size[0] * img_size[1]
if len(img_flat) == expected_size:
    # Grayscale image
    img = img_flat.reshape(img_size[0], img_size[1])
    axes[row, col].imshow(img, cmap='gray')
else:
    # Handle if image dimensions don't match
    img = img_flat[:expected_size].reshape(img_size[0], img_size[1])
    axes[row, col].imshow(img, cmap='gray')

# Get actual and predicted labels
actual = y_test[i]
predicted = y_pred_labels[i]

  # Set title with colors
 if actual == predicted:
     title_color = 'green'
    title = f"Actual: {actual}\nPredicted: {predicted} ✓"
else:
    title_color = 'red'
    title = f"Actual: {actual}\nPredicted: {predicted} ✗"

axes[row, col].set_title(title, color=title_color, fontsize=10)
axes[row, col].axis('off')
```

```
    plt.tight_layout()
    plt.show()


def display_feature_importance(model, pca, n_top_features=20):
    """
    Display feature importance from Gradient Boosting model
    """
    print("\n" + "="*50)
    print("FEATURE IMPORTANCE")
    print("="*50)

    # Get feature importance from Gradient Boosting
    feature_importance = model.feature_importances_

    # Plot top feature importances
    plt.figure(figsize=(12, 6))
    indices = np.argsort(feature_importance)[-n_top_features:]  # Top n features
    plt.barh(range(n_top_features), feature_importance[indices])
    plt.yticks(range(n_top_features), [f'PC{i+1}' for i in indices])
    plt.xlabel('Feature Importance')
    plt.title(f'Top {n_top_features} PCA Component Importances - Gradient Boosting')
    plt.tight_layout()
    plt.show()

    print(f"\nTop 5 most important PCA components:")
    top_indices = np.argsort(feature_importance)[-5:][::-1]
    for i, idx in enumerate(top_indices):
```

```
        print(f"  {i+1}. PC{idx+1}: {feature_importance[idx]:.4f}")


def main():
    """
    Main function to execute the entire pipeline
    """
    # Update this path to your actual Dataset folder location
    dataset_path = r"Dataset"  # CHANGE THIS TO YOUR ACTUAL PATH


    # First explore the deep structure to understand where images are
    explore_deep_structure(dataset_path)


    # Display skin types
    print("\n" + "="*50)
    print("DISPLAYING SKIN TYPES")
    print("="*50)
    display_skin_types(dataset_path)
    # Display skin conditions
    print("\n" + "="*50)
    print("DISPLAYING SKIN CONDITIONS")
    print("="*50)
    display_skin_conditions(dataset_path
    # Load and preprocess images (resize to 128x128 and convert to grayscale)
    print("\n" + "="*50)
    print("LOADING AND PREPROCESSING IMAGES")
    print("="*50)
```

```
# Set image parameters
IMG_SIZE = (128, 128)  # Resize to 128x128
GRAYSCALE = True        # Convert to grayscale

X, y = load_and_preprocess_images(dataset_path, img_size=IMG_SIZE, grayscale=GRAYSCALE)

if X is None or len(X) == 0:
    print(" No images loaded. Please check your dataset path and structure.")
    return

# Split data into train, validation, and test sets
(X_train, y_train), (X_val, y_val), (X_test, y_test) = split_data(X, y, test_size=0.2, val_size=0.2)

# Train Gradient Boosting classifier
gb_model, label_encoder, pca = train_gradient_boosting(
    X_train, y_train, X_val, y_val,
    n_estimators=100,       # Number of boosting stages
    learning_rate=0.1,      # Learning rate
    max_depth=3             # Maximum depth of individual trees
)

# Evaluate the model
test_accuracy, y_pred = evaluate_model(gb_model, X_test, y_test, label_encoder, pca)

# Display sample predictions
```

```
display_sample_predictions(gb_model, X_test, y_test, label_encoder, pca, img_size=IMG_SIZE, num_samples=6)


# Display feature importance
display_feature_importance(gb_model, pca, n_top_features=20)


# Additional performance metrics
print("\n" + "="*50)
print("MODEL PERFORMANCE SUMMARY")
print("="*50)
print(f"Algorithm: Gradient Boosting Classifier")
print(f"Number of estimators: 100")
print(f"Learning rate: 0.1")
print(f"Max depth: 3")
print(f"PCA components: {pca.n_components_}")
print(f"Variance explained: {pca.explained_variance_ratio_.sum()*100:.2f}%")
print(f"Test Accuracy: {test_accuracy*100:.2f}%")


print("\n Pipeline completed successfully!")


# Save the model (NOW INSIDE THE MAIN FUNCTION)
print("\n" + "="*50)
print("SAVING MODEL")
print("="*50)
try:
```

```
        with open('gradient_boosting_model.pkl', 'wb') as f:
            pickle.dump((gb_model, label_encoder, pca), f)
        print(" Model saved successfully as 'gradient_boosting_model.pkl'")
        print(f"  Model type: {type(gb_model)}")
        print(f"  Label encoder classes: {label_encoder.classes_}")
        print(f"  PCA components: {pca.n_components_}")
    except Exception as e:
        print(f" Error saving model: {e}")
    # Also save the model with a timestamp for backup
    import datetime
    timestamp = datetime.datetime.now().strftime("%Y%m%d_%H%M%S")
    backup_filename = f'gradient_boosting_model_{timestamp}.pkl'
    try:
        with open(backup_filename, 'wb') as f:
            pickle.dump((gb_model, label_encoder, pca), f)
        print(f" Backup model saved as '{backup_filename}'")
    except Exception as e:
        print(f" Could not save backup: {e}")
if __name__ == "__main__":
    main()
```

6. RESULTS AND SCREENSHOTS

6.1 Screenshots

Fig 6.1: Different Skin Types

This image presents the structured classification of different skin types utilized in the dataset for developing the skincare recommendation system, which includes combination, dry, normal, and oily skin, with approximately equal representation to maintain dataset balance and avoid bias during model training. Each category reflects distinct dermatological characteristics, where combination skin exhibits both oily and dry regions (commonly an oily T-zone and dry cheeks), dry skin is identified by rough texture, flakiness, and lack of moisture, normal skin appears smooth, clear, and well-balanced, and oily skin is characterized by excess sebum production leading to a shiny or greasy appearance. By training the machine learning model on these well-defined categories, the system can accurately classify a user's skin type based on input data or images, thereby enabling the generation of highly personalized and effective skincare recommendations tailored to individual needs.

Fig 6.2: Different Skin Conditions

This image illustrates a diverse set of skin conditions incorporated into the dataset, including acne, eczema, rosacea, milia, keratosis, and carcinoma, with each class containing a significant number of labeled images to ensure reliable training and validation of the model. These conditions display unique visual patterns, where acne is associated with pimples, blackheads, and inflammation, eczema is characterized by dry, itchy, and inflamed patches, rosacea leads to persistent facial redness and visible blood vessels, milia appear as small, white, keratin-filled cysts typically on the face, keratosis is identified by rough, scaly, or wart-like growths, and carcinoma represents abnormal or potentially cancerous skin lesions. The inclusion of these varied conditions enhances the model's ability to perform accurate classification and detection, allowing the system to not only identify skin types but also recognize underlying skin issues and provide more precise, condition-specific skincare recommendations and preventive guidance.

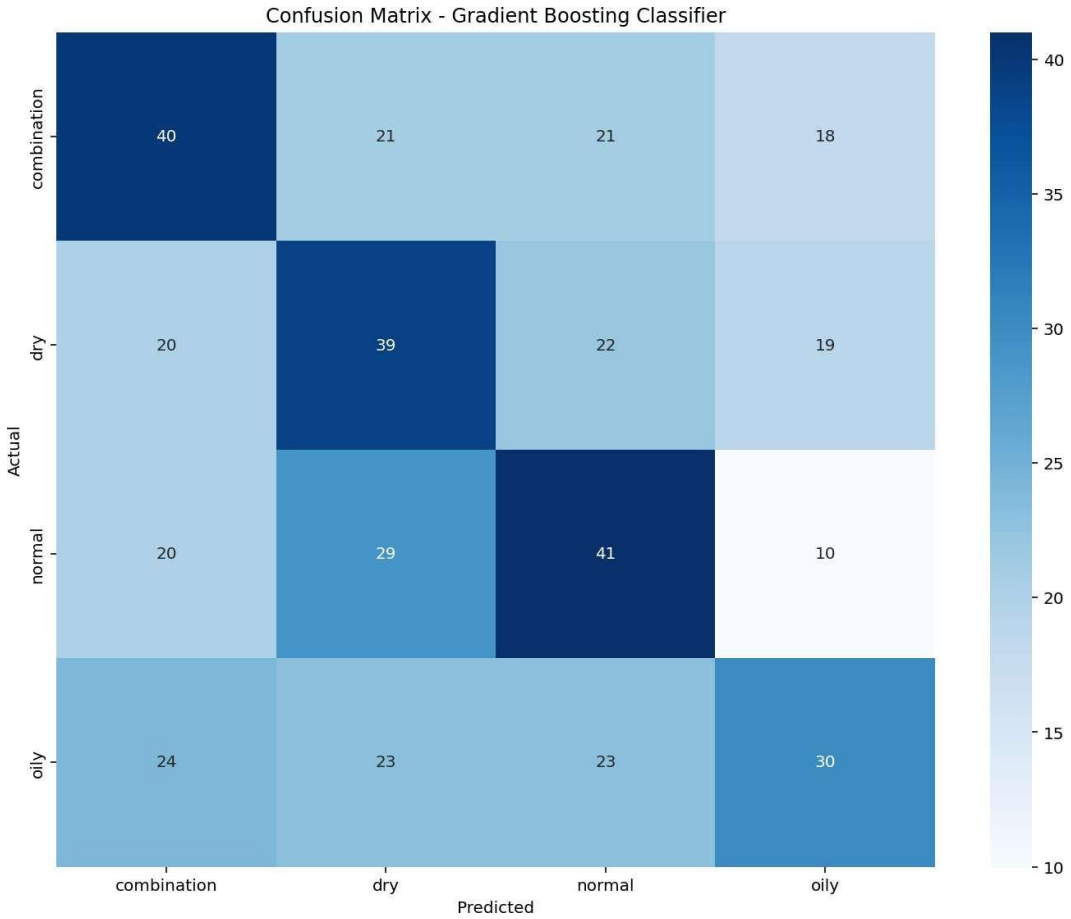

Fig 6.3: Confusion Matrix

This image represents a confusion matrix, a fundamental evaluation tool used to measure the performance of the machine learning model in classifying different skin types or skin conditions. The matrix is structured in a tabular format where the rows correspond to actual classes and the columns correspond to predicted classes. Each cell in the matrix indicates the number of instances that fall into a particular combination of actual and predicted categories. The diagonal elements of the matrix represent correctly classified instances, which contribute directly to the overall accuracy of the model, while the off-diagonal elements indicate misclassifications, highlighting where the model is making errors. By analyzing this matrix, important performance metrics such as accuracy, precision, recall, and F1-score can be derived. It also helps in identifying class imbalance issues and understanding whether certain classes (e.g., specific skin conditions) are harder to predict than others. This insight is crucial for improving the model through techniques like data augmentation, hyperparameter tuning, or model selection, ultimately enhancing the reliability of the skincare recommendation system.

Fig 6.4: Sample Predictions

This image represents the sample prediction results generated by the machine learning model using the Gradient Boosting algorithm. It visually demonstrates how the trained model processes input data (such as skin images or features) and produces accurate predictions related to skin type or skin conditions. Gradient Boosting is an ensemble learning technique that builds the model in a sequential manner by combining multiple weak learners (usually decision trees) to form a strong predictive model. Each new model focuses on correcting the errors made by the previous ones, thereby improving overall accuracy and reducing prediction errors.

The prediction output shown in the image highlights the model's ability to classify and provide results based on learned patterns from the dataset. It may include probabilities or confidence scores indicating how certain the model is about its prediction. This approach is particularly effective for complex datasets like skincare, where subtle differences in features can significantly impact the outcome. The use of Gradient Boosting enhances the system's performance by improving accuracy, robustness, and generalization capability, making the recommendations more reliable and precise.

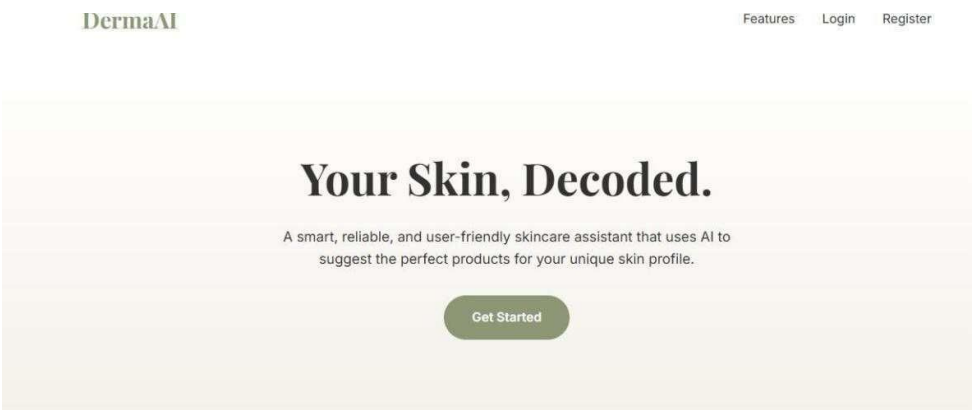

Fig 6.5: Main Page

The main page of Derma AI provides a simple and user-friendly interface introducing the platform. It highlights the purpose of the application, which is to offer AI-based skincare assistance. The page includes a clear call-to-action button ("Get Started") that allows users to begin their skincare analysis journey. The design focuses on minimalism and ease of navigation.

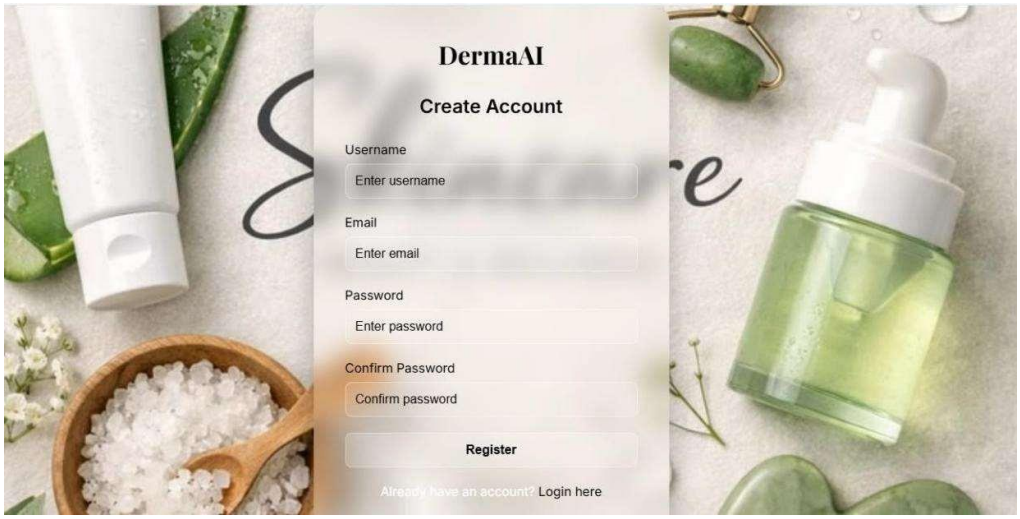

Fig 6.6: Registration Page

The registration page enables new users to create an account by entering essential details such as username, email, and password. The interface is designed with a clean layout and skincare-themed background to enhance user experience. It ensures secure access to personalized features of the system.

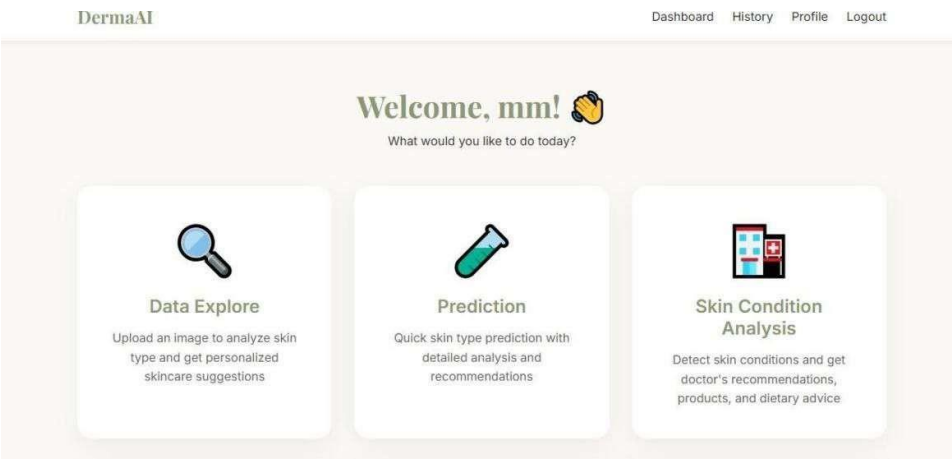

Fig 6.7: Dashboard

The dashboard acts as the central hub of the application after login. It welcomes the user and provides quick access to key features such as Data Explore, Skin Type Prediction, Skin Condition Analysis. This structured layout allows users to easily navigate and utilize different functionalities of the system.

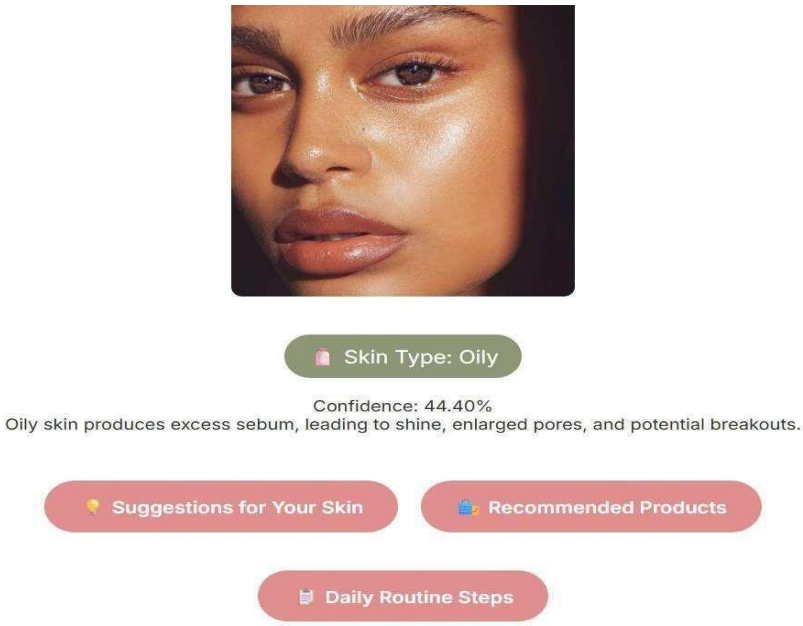

Fig 6.8: Skin Type Prediction Results

This page displays the output of the ML model after analyzing the uploaded image. It shows the detected skin type (e.g., oily) along with a confidence score. The interface also provides options to explore further recommendations such as skincare tips, product suggestions, and daily routines.

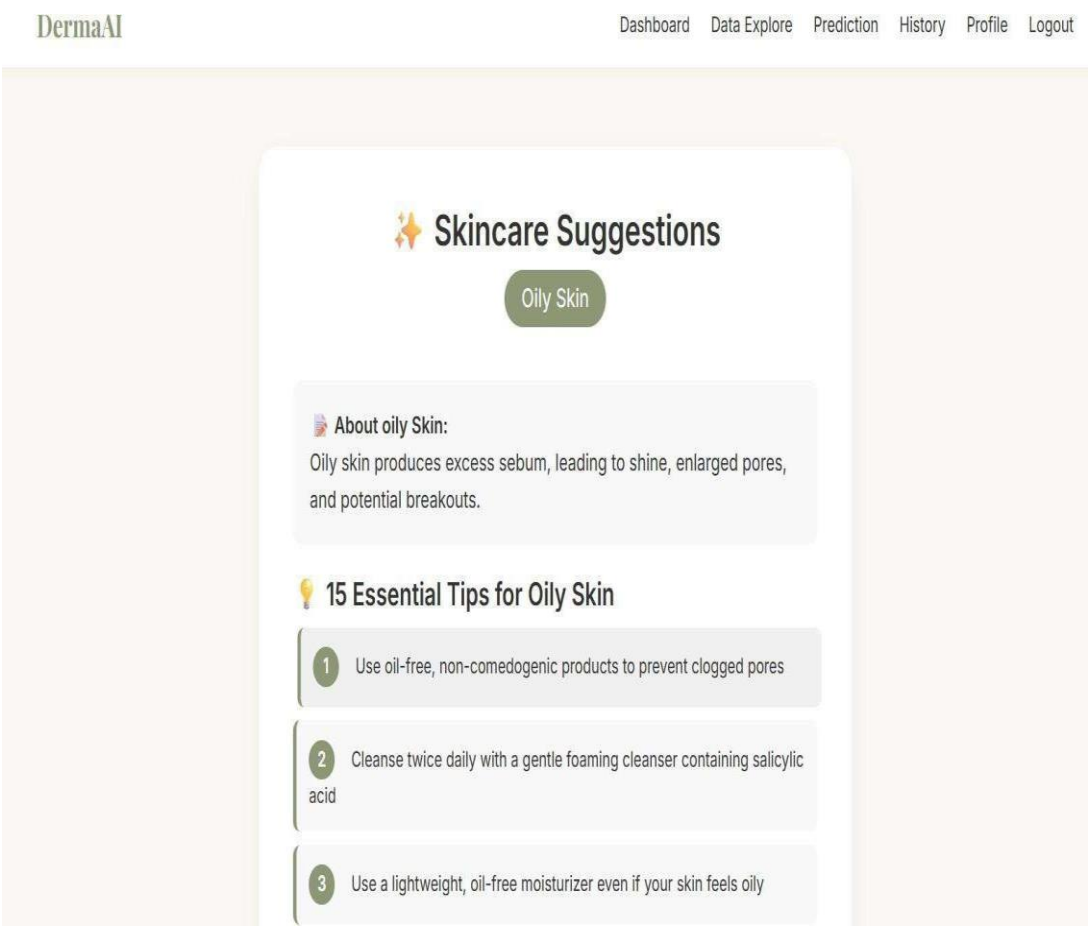

Fig 6.9: Skin Care Tips Suggestions

The skincare suggestions page provides personalized tips based on the detected skin type. It includes a brief explanation of the skin type, and a list of essential skincare tips. These suggestions help users understand their skin better and maintain healthy skincare habits.

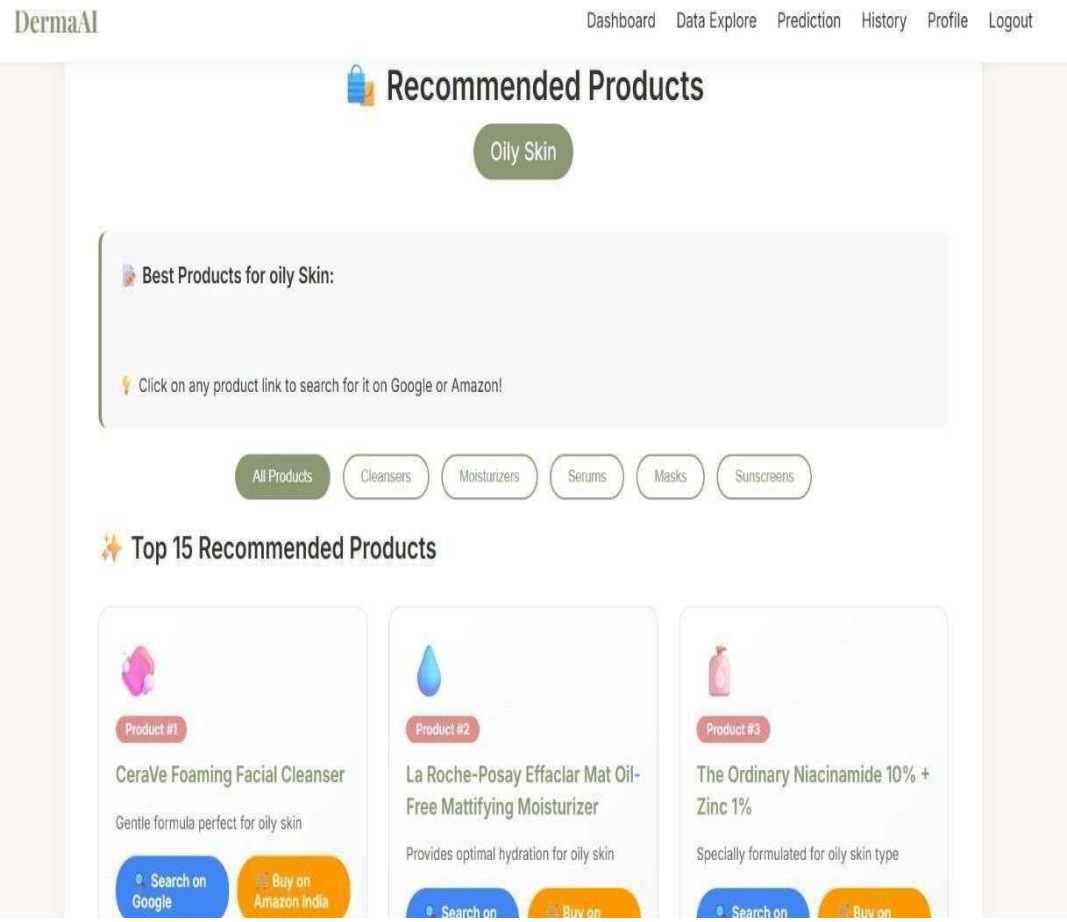

Fig 6.10: Products Recommendation

This section displays a curated list of recommended skincare products suitable for the user's skin type. Products are categorized (e.g., cleansers, moisturizers, serums), and each item includes quick access options to search or purchase online. This feature helps users choose the right products easily.

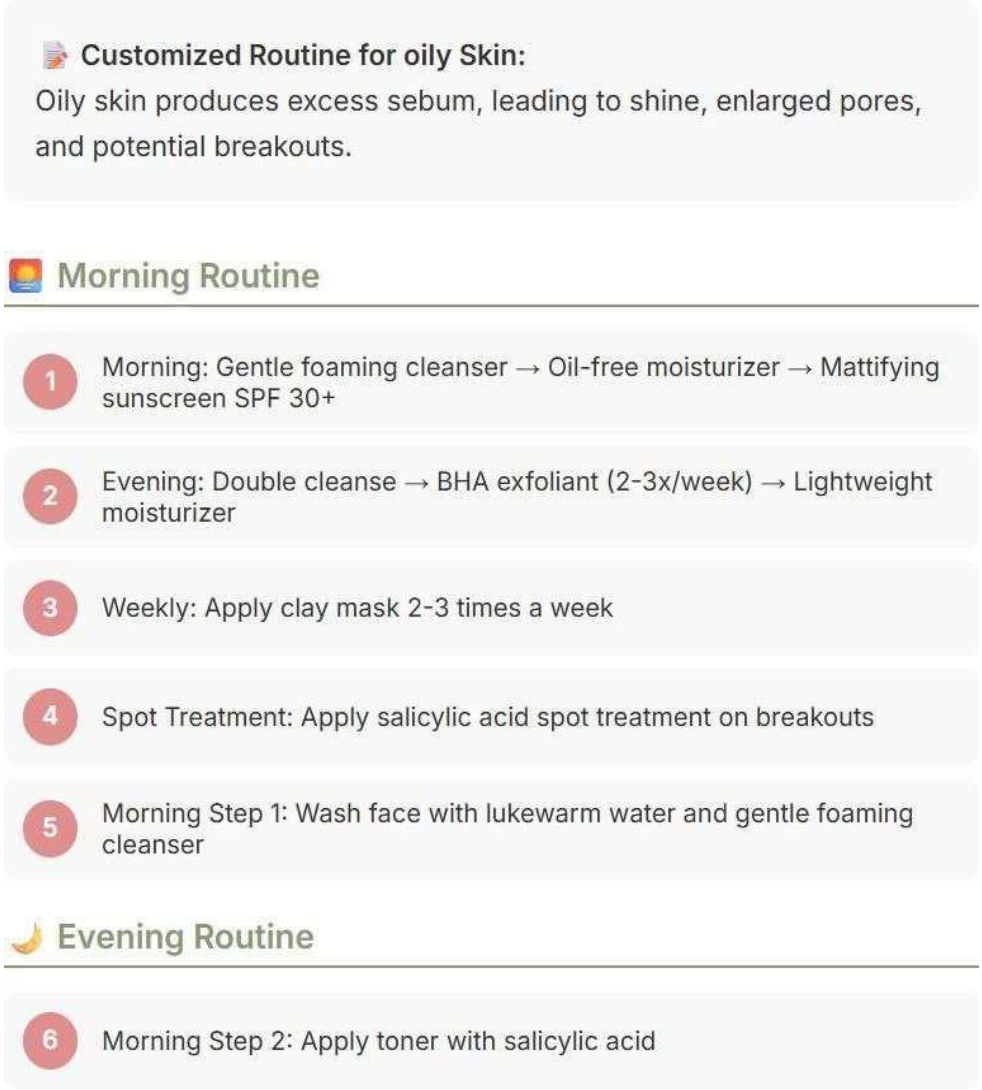

Fig 6.11: Daily Routine

The daily routine page provides a structured skincare regimen tailored to the user's skin type. It includes:

- Morning routine
- Evening routine
- Weekly care steps
- Spot treatment guidance

This ensures users can follow a consistent and effective skincare routine.

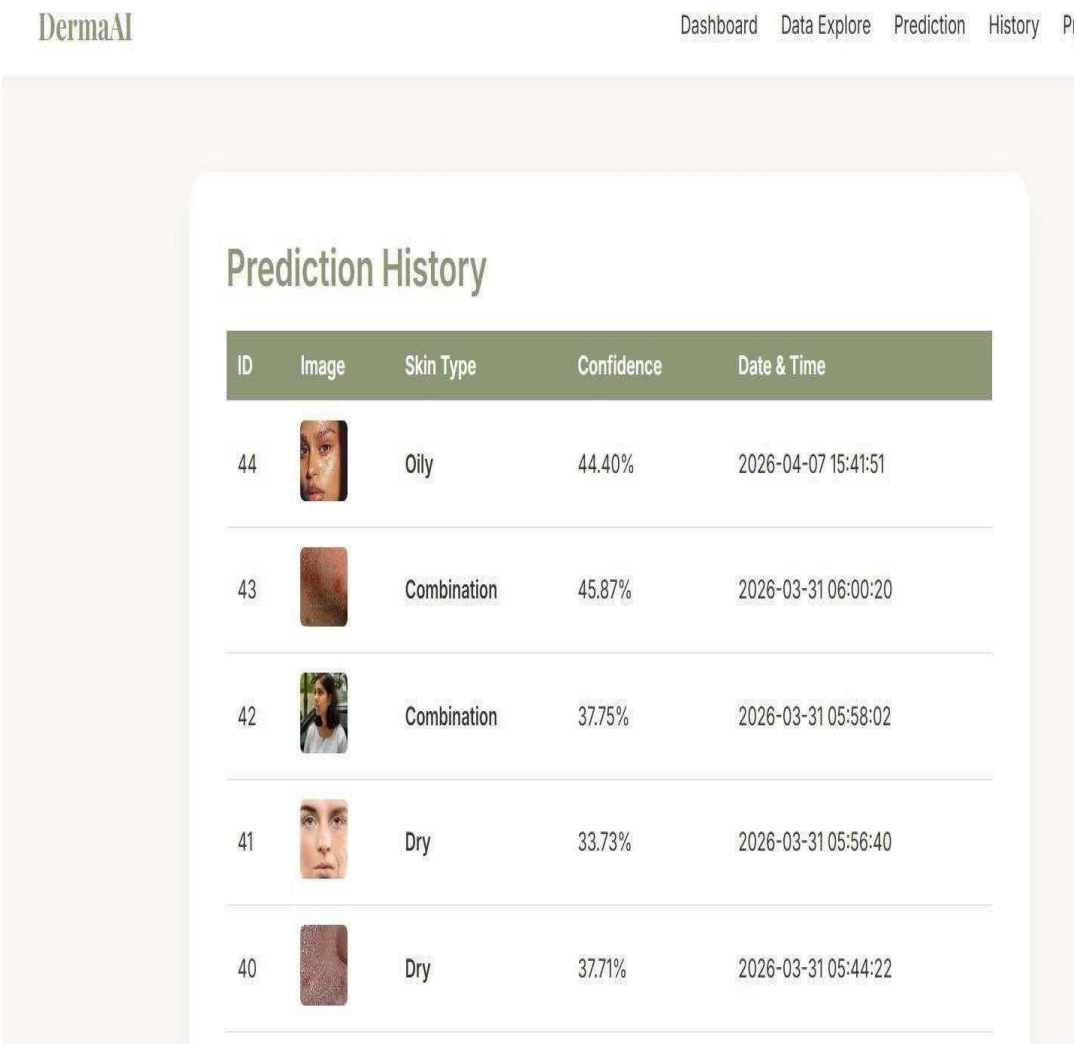

Fig 6.12: Previous Predictions

This page maintains a history of the user's past skin analyses. It displays details such as:

- Skin type detected
- Confidence level
- Date and time of prediction

This feature allows users to track changes in their skin condition over time.

6.2 Comparison with Existing Systems

The proposed skincare recommendation system offers several improvements over existing traditional and digital skincare solutions. Conventional skincare methods primarily rely on manual assessment or generalized product recommendations, which often fail to address individual skin variations and specific concerns. In contrast, the developed system leverages machine learning techniques to provide personalized and data-driven recommendations based on user input and image analysis. Existing systems, such as basic e-commerce skincare platforms, typically suggest products based on user preferences or purchase history rather than actual skin conditions. These systems lack accurate skin type detection and do not incorporate image-based analysis. On the other hand, the proposed system integrates facial image processing along with questionnaire-based inputs, enabling more precise classification of skin types and conditions.

Additionally, many traditional applications do not utilize advanced algorithms like Random Forest for prediction. The use of this algorithm in the proposed system enhances prediction accuracy, reduces overfitting, and ensures reliable results. Furthermore, the system includes multiple modules such as data exploration, prediction, skin analysis, and personalized suggestions, which are not commonly found together in existing platforms.

Another key advantage of the proposed system is its user-friendly web interface developed using Flask, which allows easy interaction and real-time response generation. Unlike some existing systems that require specialized tools or expert intervention, this application is accessible, simple to use, and does not require technical expertise.

In summary, the proposed system improves upon existing solutions by combining machine learning, image processing, and personalized recommendation techniques into a single integrated platform, offering better accuracy, usability, and efficiency while maintaining low resource requirements.

7. FUTURE ENHANCEMENTS

7.1 Future Enhancements

While the current system effectively delivers personalized skincare recommendations, there are several opportunities to further enhance its functionality, performance, and user experience. These improvements can make the application more intelligent, scalable, and user-friendly.

One major area of enhancement is machine learning advancements. The existing model can be improved by incorporating advanced algorithms such as deep learning and ensemble techniques to increase prediction accuracy. Additionally, implementing continuous learning (using user feedback and real-time data) can help the model adapt and improve over time.

Another important improvement is dataset expansion. Currently, the system relies on a limited dataset, but future versions can include larger, more diverse datasets covering various skin tones, climates, age groups, and skin conditions. This will make the recommendations more inclusive and accurate for a wider audience.

The recommendation system can also be enhanced by introducing hybrid approaches, combining content-based and collaborative filtering methods. This would allow the system to suggest products not only based on user input but also based on preferences and behaviors of similar users.

In terms of accessibility, developing a mobile application (Android/iOS) can significantly improve user reach and convenience. A mobile-friendly interface or dedicated app will allow users to access recommendations anytime, anywhere.

User experience (UX) improvements are another key area. Features like a more interactive UI, chatbot assistance, personalized dashboards, and progress tracking can make the application more engaging. Adding visual insights such as graphs or skin improvement timelines can also enhance usability.

For handling growth, scalability and performance upgrades are essential. Migrating from SQLite to more robust databases like MySQL or PostgreSQL, and deploying the application on cloud platforms can support a larger number of users and improve system reliability.

The system can also benefit from advanced analytics. By analyzing user behavior, product effectiveness, and usage patterns, the application can generate valuable insights for both users and developers, enabling better decision-making.

Future integration with external APIs and services (such as dermatology databases,

weather APIs, or product ingredient analysis tools) can further improve the accuracy and relevance of recommendations.

Ensuring data privacy and regulatory compliance is crucial. Implementing strong security measures, encryption, and compliance with data protection regulations (like GDPR) will help build user trust and safeguard sensitive information.

Finally, adding multi-language support and localization can make the application accessible to users from different regions and linguistic backgrounds, thereby increasing its global reach.

8. CONCLUSION

8.1 Conclusion

The development of this intelligent skincare assistance platform represents a significant advancement in the democratization of personalized skincare expertise. This project successfully delivers a comprehensive, Flask-based web application that integrates machine learning, secure user authentication, and holistic recommendation generation into a unified, accessible system. At the core of the application, a Random Forest classifier is trained to accurately classify four primary skin types—oily, dry, combination, and normal—from user-uploaded facial images, effectively eliminating the subjectivity and inaccuracies inherent in self-assessment methods that plague existing skincare solutions. The Python backend with Flask framework provides a robust, scalable architecture that seamlessly handles user registration, login, session management, image processing, and database operations, while the SQLite database ensures persistent storage of user credentials, skin profiles, analysis history, recommendation logs, and knowledge base content.

8.2 Future Work

While the current implementation successfully achieves its core objectives, numerous opportunities exist to expand, refine, and enhance the application's capabilities. These future enhancements span multiple dimensions including machine learning advancements, dataset expansion, recommendation system improvements, mobile development, user experience enhancements, scalability upgrades, advanced analytics, external integrations, regulatory compliance, and language localization. The modular architecture of the existing system ensures that these enhancements can be incrementally implemented without disrupting core functionality.

9. REFERENCES

[1] Jannatin, A. A., & Ratnasari, C. I. (2025). Personalized Skincare Recommendation System Based on Ontology and User Preferences. bit-Tech, 8(1), 1109-1119.

[2] Crocker Garcia, G., Khan, M. N., Alam, A., Obregon, J., Abuhmed, T., & Huh, E. N. (2025). FiSC: A Novel Approach for Fitzpatrick Scale-Based Skin Analyzer's Image Classification. IEEE Access, 13, 42934-42948.

[3] Malaysian Facial Skin Texture Dataset (MFSTD) Development. (2025). In Proceedings of the International Conference on Image Processing and Computer Vision. IEEE.

[4] Deep Neural Network-Based Personalized Makeup Recommendation Using Korean Skin Physiological Indicators. (2025). Journal of the Korean Society of Cosmetology.

[5] RecomiGlowTech: Cosmetic Recommendation System. (2025). Informatics Institute of Technology, Sri Lanka.

[6] Ng, Y. S. (2025). Skin Analysis and Recommendation System (Undergraduate Final Year Project). Universiti Tunku Abdul Rahman, Malaysia.

[7] Skinwise: Skincare Product Management & Recommendation Platform – An AI-Powered Platform Integrating Computer Vision and a Dermatology-Informed Safety Engine. (2025). ScienceOpen Preprints. Amal Jyothi College of Engineering.

[8] Baumann, L. (2015). The Baumann Skin Typing System. In Cosmetic Dermatology: Principles and Practice (2nd ed.). McGraw-Hill Education.

[9] Draelos, Z. D. (2019). Cosmeceuticals: A Comprehensive Guide to Skin Care Products. Elsevier.